

Try Except در پایتون برای مدیریت خطا

از ساختار **Try Except** در پایتون برای مدیریت خطاهای استثنای استفاده می‌شود. این نوع از خطاها در شرایط خاص و غیرعادی هنگام اجرای برنامه رخ می‌دهند. خود برنامه باید توانایی شناسایی و کنترل این حالت‌ها را داشته باشد. برنامه نویس با پیش‌بینی حالت‌های استثنا مختلف، عکس‌العمل مناسب را برای مدیریت خطا تعریف می‌کند. برنامه نویسان پایتون از بلوک‌های **Try Except** برای مدیریت خطاهای استثنا استفاده می‌کنند. استفاده از ساختار **Try Except** در پایتون از خرابی برنامه یا کرش کردن سیستم در زمان دادن خطا جلوگیری می‌کند. این ابزارها به برنامه نویس کمک می‌کنند که خطاهای اعلام شده توسط مفسر یا سیستم عامل را شناسایی و مدیریت کنند. در نتیجه کدهای نوشته شده از استحکام بیشتری برخوردار شده و برنامه هم کاربرپسندتر می‌شود.

سینتکس Try Except در پایتون

برای مدیریت خطاهای استثنا در پایتون از بلوک‌های **try** و **except** و **else** و **finally** استفاده می‌کنیم. در کادر پایین ترتیب قرار دادن این بلوک‌ها را نوشته‌ایم.

```
1 try:
2     # Code that might raise an exception
3 except SomeException:
4     # Code to handle the exception
5 else:
6     # Code to run if no exception occurs
7 finally:
8     # Code to run regardless of whether an exception occurs
```

- **بلوک try:** در بلوک `try` کد اصلی برنامه را می‌نویسیم. این کد ممکن است در زمان اجرا با یکی از حالت‌های استثنا روبه‌رو شود. کلمه «`try`» به معنی تلاش کردن است. از آن‌جا که پایتون تلاش می‌کند کدهای نوشته شده در این بلوک را اجرا کند نام این بلوک `try` گذاشته شده است. در صورت رویدادن استثنا و بروز خطا، فرایند اجرای کدها بلافاصله به بلوک `except` منتقل می‌شود.
- **بلوک except:** این بلوک به برنامه نویسان برای مدیریت حالت‌های استثنا کمک می‌کند. اگر کدهای نوشته شده در بلوک `try` با خطای استثنایی روبه‌رو شوند، پایتون ادامه اجرای کدها را به بلوک `except` منتقل می‌کند. اکنون نوبت اجرای کدهای درون بلوک `except` است. برنامه نویسان می‌توانند استثناهای خاص را به صورت جداگانه مدیریت کنند. همچنین امکان استفاده از یک بلوک کلی برای دریافت و کنترل تمام خطاهای استثنایی نیز وجود دارد. این روش را در بخش‌های بعدی مطلب با کمک مثال بررسی کرده‌ایم.
- **بلوک else:** استفاده از بلوک `else` اختیاری است. اما اگر از این بلوک استفاده کردیم باید بعد از همه بلوک‌های `except` قرار بگیرد. بلوک `else` فقط وقتی اجرا می‌شود که بلوک `try` با هیچ خطا و حالت استثنایی روبه‌رو نشده باشد. گاهی از اوقات می‌خواهیم که اگر کدهای بلوک `try` بخوبی و بدون مشکل اجرا شدند، دسته‌ای از کدهای دیگر هم اجرا شوند. استفاده از بلوک `else` روش مناسبی برای اجرای آن دسته از کدها است.
- **بلوک finally:** کدهای نوشته شده در بلوک `finally` همیشه اجرا می‌شوند. بدون در نظر گرفتن اینکه آیا حالت استثنایی رخ داده است یا نه. به طور کلی از این کدها برای اجرای «عملیات پاکسازی» (`Cleanup Operations`) استفاده می‌کنیم. عملیات پاک‌سازی به کارهایی مانند بستن فایل‌ها یا آزاد کردن منابع گفته می‌شود.

```

1 try:
2     n = 0
3     res = 100 / n
4
5 except ZeroDivisionError:
6     print("You can't divide by zero!")
7
8 except ValueError:
9     print("Enter a valid number!")
10
11 else:
12     print("Result is", res)
13
14 finally:
15     print("Execution complete.")

```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نشان داده می‌شود.

```

You can't divide by zero!
Execution complete.

```

مثالی از مدیریت خطای ساده در پایتون

به کار بردن تکنیک‌های مدیریت خطا در پایتون از کرش کردن برنامه‌ها در زمان اجرا جلوگیری می‌کند. در این بخش از برنامه، مثال ساده‌ای از بروز خطاهای کوچک و روش دریافت و مدیریت صحیح آن‌ها را بررسی کرده‌ایم.

```

1 # Simple Exception Handling Example
2 n = 10
3 try:
4     res = n / 0 # This will raise a ZeroDivisionError
5
6 except ZeroDivisionError:
7     print("Can't be divided by zero!")

```

بعد از اجرای کدهای بالا و بروز خطا، خروجی زیر در کنسول پایتون به نمایش در می‌آید.

```

Can't be divided by zero!

```

تفاوت بین استثنا و خطا در پایتون چیست؟

- **خطا (Error):** خطا به مشکلات جدی گفته می‌شود. برنامه‌ها نباید این مشکلات را مدیریت کنند. خطا معمولاً مشکلی در منطق کد یا پی‌کربندی برنامه است. خود برنامه نویس باید خطاها را تعمیر و رفع کند. به عنوان مثال‌هایی از این دسته خطاها می‌توانیم به خطای مربوط به سینتکس یا مدیریت حافظه اشاره کنیم.
- **استثنا (Exception):** شدت مشکل بوجود آمده توسط استثنا کمتر از خطا است. در نتیجه خود برنامه‌ها هم می‌توانند استثناها را مدیریت کنند. این نوع از مشکلات ممکن است در موقعیت‌های مختلفی پیش بیایند. برای مثال اگر کاربری داده‌های ورودی نامعتبر وارد کند. یا فایل‌هایی در برنامه گم شوند. یا مشکلاتی در شبکه به وجود بیایند.

بلوک try چگونه کار می‌کند؟

بلوک try را برای بررسی کدهایی به کار می‌بریم که احتمال بروز خطا در آن‌ها وجود دارد، یعنی کدهای اصلی برنامه. اگر هیچ خطایی در برنامه نباشد، فقط کدهای نوشته شده در بلوک try اجرا می‌شوند. در حالی که اگر کدهای بلوک try با خطای استثنا روبه‌رو شوند، کدهای درون بلوک except اجرا می‌شوند.

```
1 # Python code to illustrate
2 # working of try()
3 def divide(x, y):
4     try:
5         # Floor Division : Gives only Fractional Part as Answer
6         result = x // y
7         print("Yeah ! Your answer is :", result)
8     except ZeroDivisionError:
9         print("Sorry ! You are dividing by zero ")
10
11 # Look at parameters and note the working of Program
12 divide(3, 2)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نشان داده می‌شود.

```
Yeah ! Your answer is : 1
```

در کدهای زیر، روش دیگری از نوشتن بلوک `except` را بررسی کرده‌ایم. در این حالت بلوک `except` فقط خطاهایی را دریافت و مدیریت می‌کند که برنامه نویسی کرده‌ایم. این تکنیک به توسعه‌دهندگان کمک می‌کند تا بفهمند چه خطایی رخ داده است.

```
1 # code
2 def divide(x, y):
3     try:
4         # Floor Division : Gives only Fractional Part as Answer
5         result = x // y
6         print("Yeah ! Your answer is :", result)
7     except Exception as e:
8         # By this way we can know about the type of error occurring
9         print("The error is: ",e)
10
11
12 divide(3, "Faradars")
13 divide(3,0)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نشان داده می‌شود.

```
The error is: unsupported operand type(s) for //: 'int' and 'str'
The error is: integer division or modulo by zero
```

بلوک Else چگونه کار می‌کند؟

برنامه نویسان پایتون می‌توانند از بلوک `else` در ادامه ساختار `Try Except` استفاده کنند. بلوک `else` باید بعد از تمام بلوک‌های `except` قرار بگیرد. کدهای درون بلوک `else` فقط به شرطی اجرا می‌شوند که در زمان اجرای کدهای `try` هیچ خطایی رخ نداده باشد. در پایین، روش نوشتن این بلوک را نشان داده‌ایم.

```
1 def AbyB(a , b):  
2     try:  
3         c = ((a+b) // (a-b))  
4     except ZeroDivisionError:  
5         print ("a/b result in 0")  
6     else:  
7         print (c)  
8  
9 # Driver program to test above function  
10 AbyB(2.0, 3.0)  
11 AbyB(3.0, 3.0)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نشان داده می‌شود.

```
-5.0  
a/b result in 0
```

بلوک Finally چگونه کار می‌کند؟

در پایتون، کلمه کلیدی با عبارت **finally** تعریف شده است. از این کلمه کلیدی برای تعریف بلوک خاصی از کدها استفاده می‌کنیم. این بلوک بعد از بلوک‌های **try** و **except** و **else** قرار می‌گیرد. تفاوتی نمی‌کند که کدهای بلوک **try** با موفقیت اجرا شده باشند یا به دلیل بروز خطا برنامه متوقف شده باشد. کدهای بلوک **finally** همیشه اجرا می‌شوند.

```
1 try:
2     k = 5//0 # raises divide by zero exception.
3     print(k)
4
5 # handles zerodivision exception
6 except ZeroDivisionError:
7     print("Can't divide by zero")
8
9 finally:
10    # this block is always executed
11    # regardless of exception generation.
12    print('This is always executed')
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نشان داده می‌شود.

```
Can't divide by zero
This is always executed
```

رایج‌ترین حالت‌های استثنا در پایتون

- **BaseException**: این حالت، کلاس پایه‌ای است که تمام خطاهای استثنای «درونی (Built-In)» پایتون از آن مشتق شده‌اند.
- **Exception**: کلاس پایه برای تمام خطاهای استثنایی است که باعث توقف برنامه نمی‌شوند.
- **ArithmeticError**: کلاس پایه برای تمام خطاهای استثنایی که به عملیات ریاضی مربوط می‌شوند.
- **ZeroDivisionError**: این خطای استثنا وقتی اعلام می‌شود که در عملیات [تقسیم در پایتون](#) «//» یا باقیمانده صحیح «%» مقسوم‌علیه صفر باشد.
- **OverflowError**: این حالت، زمانی اعلام می‌شود که نتیجه عملیات عددی از بیشینه مقدار در آن نوع داده، بیشتر شود.

- **FloatingPointError:** هر وقت عملیات مربوط به اعداد اعشاری با شکست روبه‌رو شود این حالت پیش می‌آید.
- **AssertionError:** زمانی رخ می‌دهد که دستور `assert` با شکست مواجه شود.
- **AttributeError:** این حالت زمانی روی می‌دهد که ارجاع یا تخصیص ویژگی با شکست روبه‌رو شود.
- **IndexError:** زمانی روی می‌دهد که در یک عملیات به اندیس خارج از محدوده در دنباله - مانند لیست یا تاپل - اشاره کنیم.
- **KeyError:** وقتی اعلام می‌شود که کلید مورد نظر در دیکشنری پیدا نمی‌شود.
- **MemoryError:** وقتی این حالت رخ می‌دهد که عملیات محاسباتی خاصی با کمبود حافظه روبه‌رو شود.
- **NameError:** وقتی که نام محلی یا جهانی پیدا نشود این حالت استثنا رخ می‌دهد.
- **OSError:** اگر یکی از وظایف سیستمی مانند خواندن یا نوشتن اطلاعات بر روی فایل‌ها با شکست روبه‌رو شود این خطا روی می‌دهد.
- **TypeError:** این خطا زمانی روی می‌دهد که از توابع بر روی داده‌های با نوع اشتباه استفاده کنیم.
- **ValueError:** این حالت وقتی اتفاق می‌افتد که تابعی به درستی فراخوانی شده است. اما داده‌های ارسال شده به آن - به عنوان پارامتر - غلط باشند.
- **ImportError:** این خطا زمانی روی می‌دهد که در عبارت `import` اشتباهی انجام داده باشیم.
- **ModuleNotFoundError:** این خطای استثنا وقتی اعلام می‌شود که مفسر پایتون موفق به پیدا کردن ماژول مورد نظر نشود.

اعلام کردن حالت استثنا

در زبان برنامه نویسی پایتون برای ایجاد حالت استثنا از کلمه کلیدی `raise` استفاده می‌کنیم. ابتدا کلمه کلیدی `raise` را نوشته و بعد از آن هم شیئی از کلاس استثنای مورد نظر را در ادامه آن قرار می‌دهیم. اعلام کردن حالت استثنا یعنی آنکه در وضعیت سفارشی و دلخواهی به جای ادامه اجرای عادی برنامه، خطای مشخصی ایجاد کنیم. در این حالت می‌توانیم از استثناهای پیش‌فرض پایتون استفاده کنیم.

```
1 def set(age):
2     if age < 0:
3         raise ValueError("Age cannot be negative.")
4     print(f"Age set to {age}")
5
6 try:
7     set(-5)
8 except ValueError as e:
9     print(e)
```

بعد از اجرای کدهای بالا، خروجی زیر تولید شده و در کنسول پایتون نشان داده می‌شود.

```
Age cannot be negative.
```

مزایای استفاده از Try Except در پایتون

- **افزایش خوانایی برنامه:** با مدیریت مناسب خطاهای استثنا می‌توانیم از کرش کردن برنامه‌ها جلوگیری کنیم. با استفاده از این ساختار می‌توانیم از بوجود آمدن نتایج غلط بر اثر خطاهای ناخواسته یا داده‌های ورودی اشتباه هم پیشگیری کنیم.
- **ساده‌سازی عملیات مدیریت خطا:** با استفاده از بلوک‌های `Try Except` در پایتون، بخش مربوط به مدیریت خطا از بدنه اصلی کدها جدا می‌شود. در نتیجه درک و خواندن منطق برنامه و کدها ساده‌تر شده و نگهداری آن‌هم راحت‌تر می‌شود.
- **نوشتن کدهای تمیزتر:** به کمک تکنیک‌های مدیریت استثنا در پایتون، برنامه نویسان از به کار بردن عبارت‌های شرطی پیچیده برای بررسی خطا بی‌نیاز می‌شوند. در نتیجه، کدها ساده‌تر، خواناتر و حرفه‌ای‌تر نوشته می‌شوند.
- **دیباگ کردن ساده‌تر کدها:** وقتی که خطای استثنایی اعلام می‌شود، [مفسر پایتون](#) بر روی کنسول پیغامی چاپ می‌کند. این پیغام، محل دقیق وقوع خطای استثنا را نشان می‌دهد. در نتیجه اشکال‌زدایی کد آسان‌تر می‌شود.

معایب استفاده از Try Except در پایتون

- **عملکرد کندتر:** استفاده از بلوک‌های Try Except در پایتون برای بررسی خطاها نسبت به عبارت if کندتر است. این مسئله بخاطر آن است که مفسر پایتون برای شناسایی و مدیریت استثناها باید عملیات بیشتری انجام دهد.
- **افزایش پیچیدگی کدها:** استفاده زیاد از ساختارهای مدیریت خطا ممکن است خوانایی کد را کاهش دهد. این مشکل به طور خاص زمانی اتفاق می‌افتد که توسعه‌دهنده بخواهد چند نوع خطای مختلف را مدیریت کرده یا از قوانین پیچیده‌ای استفاده کند.
- **احتمال وقوع خطرهای امنیتی:** اگر خطاهای استثنا را به درستی مدیریت نکنیم، احتمال دارد اطلاعات محرمانه‌ای درباره کدها در کنسول به نمایش گذاشته شود. در نتیجه امنیت برنامه به خطر می‌افتد. بنابراین مدیریت صحیح حالت‌های استثنا از اهمیت بسیار زیادی برخوردار است. نباید اطلاعات اضافی درباره برنامه و عملکرد آن نمایش داده شود.